임기호

Backend Engineer | Node.js · TypeScript · PostgreSQL

현업의 복잡한 업무와 부서별 데이터 기준을 분석해 하나의 플랫폼으로 통합하고, 반복 작업을 자동화해 왔습니다.
현재는 3인 개발팀에서 항공·해운·재무 통합 플랫폼의 구조 설계와 주요 백엔드 개발을 담당하고 있습니다.

KWE Korea | 차장 | 2022.01 – 현재

lasid84@gmail.com · GitHub · 상세 경력기술서 · 프로젝트 포트폴리오

---

01 분산된 업무 시스템을 사내 핵심 플랫폼으로 통합

15종 이상 개별 시스템 → **사내 핵심 업무 플랫폼**

Node.js · TypeScript · Next.js · PostgreSQL · Turborepo

항공·해운·재무 업무에는 VB6·C#·React·Oracle Forms 등 서로 다른 기술로 개발된 15종 이상의 시스템이 사용되고 있었습니다. 일부 시스템은 잦은 연결 끊김과 불편한 사용 환경으로 업무에 지장을 주었고, 기능과 데이터도 여러 시스템에 나뉘어 있었습니다. 이러한 문제를 개선하기 위해 하나의 웹 플랫폼으로 통합을 추진했습니다.

통합 과정에서는 부서마다 다른 기준정보를 맞추는 일이 중요했습니다. 같은 고객도 부서에 따라 다른 거래처로 관리돼 기존 고객에게 서류를 다시 요청하는 문제가 있었습니다. 사업자번호와 본사 코드를 기준으로 거래처 데이터를 대조하고, 현업과 예외 사항을 검증해 공통 식별 기준을 마련했습니다.

기존 기능과 업무 흐름을 현업과 함께 분석해 유지할 기능과 재설계할 업무를 구분했습니다. 거래처·항공사 등의 기준정보를 표준화하고, 항공·해운·재무의 주요 업무와 데이터를 하나의 웹 플랫폼으로 통합했습니다. 이를 통해 서로 다른 프로그램에 분산돼 있던 업무를 공통 플랫폼에서 처리할 수 있게 됐습니다. 현재는 항공·해운·재무 업무를 수행하는 사내 핵심 시스템으로 운영되고 있습니다.

02 반복 입력 자동화와 사내 회선 사용량 절감

Puppeteer → HTTP API | **60~70Mbps → 5Mbps 미만**

Node.js · TypeScript · Puppeteer · HTTP API · Redis

현업에서는 내부 시스템에 입력한 정보를 해외 본사 업무 시스템(UFS)에도 다시 입력해야 했습니다. 본사 시스템은 입력 절차가 복잡하고 국내 업무에 필요한 편의 기능도 부족했습니다.

처음에는 본사 시스템의 변경에 대응하기 쉽다고 판단해 Puppeteer로 조회·입력을 자동화했습니다. 그러나 화면 자원 로딩으로 네트워크 사용량이 증가했습니다. 사내 Grafana에서는 최대 60~70Mbps가 관측됐으며, 이는 100Mbps 사내 회선 용량의 최대 약 70%에 해당했습니다.

불필요한 화면 로딩으로 발생하는 트래픽을 줄이기 위해 인증 과정과 HTTP 요청 구조를 분석해 동료 1명과 API 호출 75개를 구현했습니다. 12종 업무 흐름을 HTTP 방식으로 전환한 후 네트워크 사용량은 5Mbps 미만으로 확인됐습니다.

03 통관 서류 검토와 후속 업무 자동화

통관 1건 검토 **8분 → 3분** | 후속 업무 하루 **약 1시간** 추가 절감

Tesseract.js · Local LLM · 문서 자동화

통관 1건당 통상 9종 서류의 55개 항목을 담당자가 직접 대조해야 했습니다.

문서 형식에 따라 추출한 텍스트·원본 파일·OCR 결과를 활용하고, 항목별 설명과 Few-shot 예시로 LLM 추출 기준을 정의했습니다. 추출 결과를 바탕으로 서류 간 누락·불일치를 검토하도록 구현했습니다.

이를 통해 건당 검토 시간을 8분에서 3분으로 줄였습니다. 이후 시스템 등록·관리와 회신 메일 작성·파일 첨부까지 자동화해 후속 업무에서도 하루 약 1시간을 추가 절감했습니다.

백엔드 설계 및 운영

운영 중 발견한 문제를 분석하고, 지속적으로 개선해 온 백엔드 설계 경험

04 비동기 작업 처리와 분산 환경의 동시성 제어

Redis 세션 공유 · 분산 락을 통한 중복 인증 방지

Node.js · NestJS · BullMQ · Redis

사용자 요청과 자동 배치에서 동일한 업무 로직을 재사용할 수 있도록 비즈니스 로직을 API에 일원화하고, BullMQ Worker는 작업 실행과 재시도를 담당하도록 분리했습니다.

API 서버 2개 레플리카에서 해외 본사 업무 시스템의 로그인 세션을 Redis로 공유했습니다. 분산 락으로 동시 인증을 제어하고, 락 대기 중 다른 요청이 갱신한 세션을 재사용해 중복 인증을 방지했습니다. 토큰 저장과 락 해제 시에는 락 소유권을 검증했습니다.

05 Node.js 메모리 장애 원인 분석 및 개선

운영 로그 재분석 · Heap/RSS 측정 · 대량 데이터 처리 구조 개선

Node.js · Prisma · PostgreSQL

운영 중 Node.js API 서버가 메모리 부족으로 반복 종료됐습니다. 처음에는 OCR 처리와 Prisma 쿼리 엔진을 의심했지만 관련 조치 후에도 장애가 이어졌습니다.

실제 배포 환경과 실행 로그, 조회 조건을 다시 분석해 수만 건의 데이터를 한 번에 조회·가공하는 경로를 확인했습니다. 서버에서는 500건 단위로 조회하도록 바꾸고 화면에는 무한스크롤을 적용했습니다.

조회 구조를 개선한 뒤, 별도 모사 실험에서 응답 데이터의 가공·직렬화 과정에 따른 Heap과 RSS 변화를 측정했습니다. 반복 실행에서도 GC 이후 힙 사용량이 지속적으로 누적되는 현상은 확인되지 않아, 일시적인 메모리 증가와 지속적인 누수 가능성을 구분해 검토했습니다. 조회 구조 개선 이후 동일 장애는 재발하지 않았습니다.

06 개발 품질과 배포 환경 개선

전체 파이프라인 **12분 → 6분**

GitLab CI/CD · Docker · buildx · Vitest

테스트 실패 시 후속 빌드와 배포를 차단하도록 CI/CD를 구성했습니다. Kaniko에서 buildx로 전환하고 러너·캐시 구성을 개선해 전체 파이프라인 시간을 약 12분에서 6분으로 줄였습니다.

직접 기획하고 출시한 제품

댓글 필터 · 헬스독 · 액션독

React Native·Expo를 활용해 앱 3종을 직접 기획·개발하고 출시했습니다.

액션독 출시 후 초기 다운로드 현황을 검토해 사용자 유입 확보를 우선 과제로 정하고, 리그·수익화 기능은 후순위로 조정했습니다. 설치 부담을 줄이기 위해 불필요한 Android 권한을 제거하고, 플레이 경험 개선을 위해 게임 재시작 화면을 정비했습니다.

이전 경력

일진글로벌 | 2010.12 – 2022.01

MES 개발·운영 · C#/.NET · Oracle · SAP 연동

상세 구현 과정 및 프로젝트 · 전체 경력기술서 · 프로젝트 포트폴리오 · GitHub